2026

Платформы и среды выполнения ЯП

NEXIGN x ИТМО

Раздел 01

Жизненный цикл программы и модели исполнения



Компоновка в схеме трансляции

Общая схема трансляции:
Исходный код → препроцессор → компилятор → ассемблер → объектный файл → **компоновщик** → исполняемый файл → загрузчик → процесс в памяти.

- На входе компоновщика: секции, таблица символов и записи релокаций.
- Часть ссылок пока не разрешена: например, вызов из другого модуля.
- Две задачи: **разрешение символов** и **релокация (перемещение)**.

Символы объектного файла

Глобальные
определения, доступные другим единицам трансляции

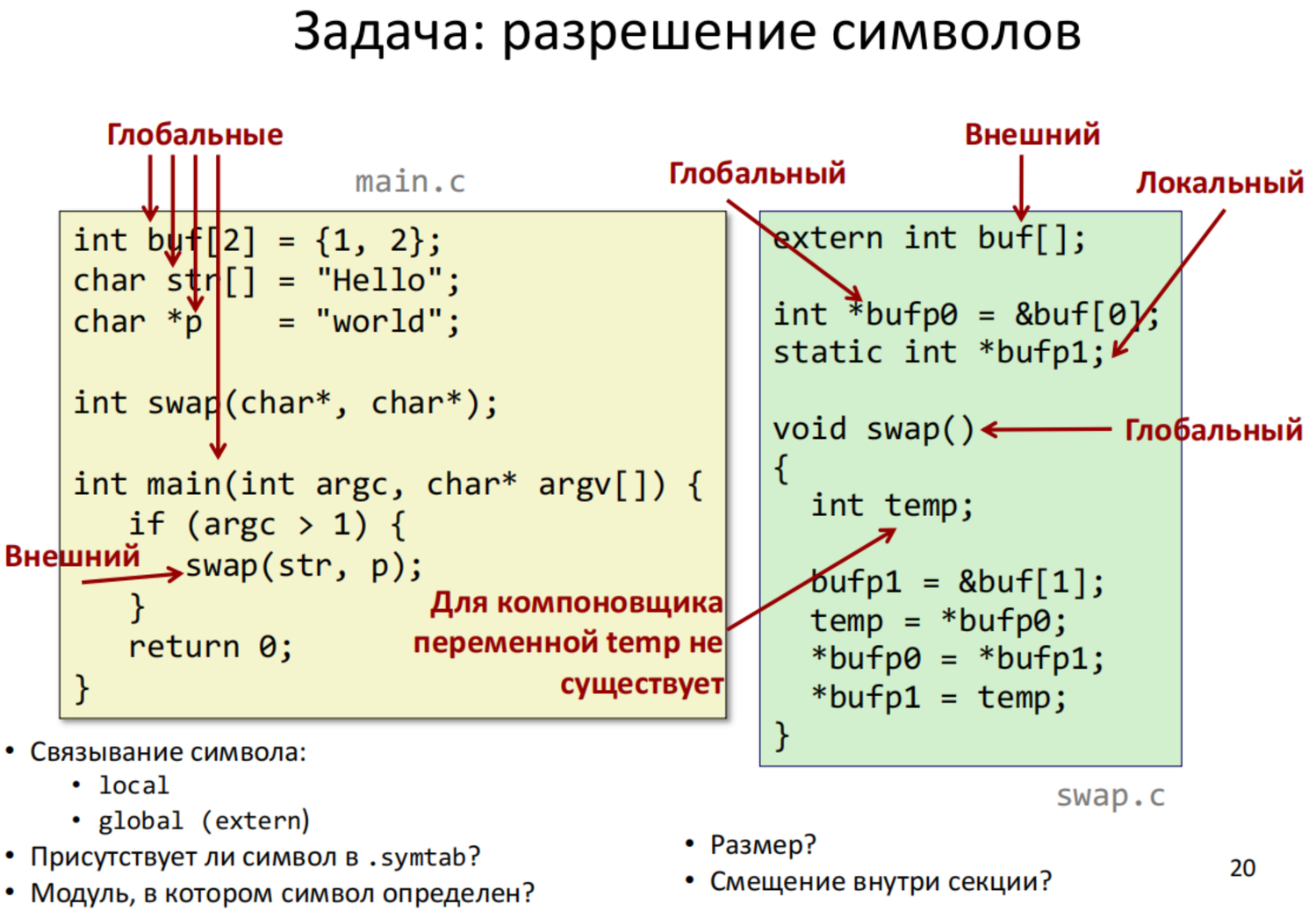
Локальные
определены и используются в одном и том же модуле

Неопределенные
символы, для которых в данном .o нет определения

локальная переменная функции ≠ локальный символ компоновщика

Символы объектного файла

Источник:
http://asmcourse.cs.msu.ru



Strong, weak и common символы

Механизм	Представление / смысл
Strong definition	Функции и инициализированные глобальные переменные
Weak definition	Неинициализированные глобальные переменные (STB_WEAK)
Common symbol	SHN_COMMON, типичный результат -fcommon для tentative definition в C

Правила разрешения символов

Для GNU ELF-компоновки:

1. Несколько сильных определений одного символа обычно приводят к ошибке.
2. Сильное определение имеет приоритет над слабым.
3. При нескольких слабых выбор определяется правилами компоновщика.
4. Common-символы обрабатываются отдельно; несколько common-определений могут объединяться, а сильное определение имеет приоритет.

Пример

```c
// module1.c
int y = 15212;
int x = 15213;
```

```c
// module2.c
double x;
void f(void) { x = -0.0; }
```

- Исторический сценарий: несовместимые определения x могут быть объединены.
- Современный сценарий: зависит от платформы, однако успешная линковка не гарантирует корректности программы.

- В худшем случае: x = 0, y = 0x80000000

Пример: что произошло

1. **module1.c:**
   - int y = 15212; — инициализированная переменная → Сильный символ.
   - int x = 15213; — инициализированная переменная → Сильный символ.

2. **module2.c:**
   - double x; — неинициализированная переменная → Слабый символ.

3. Одно сильное и несколько слабых определений → **Выбирается сильное**.

4. Вызывается функция f() из module2.c. **Компилятор сгенерировал для нее код, исходя из того, что x — это double (8 байт)**.

5. **:(**

Дальше больше: name mangling в C++

Name Mangling and extern "C" in C++

Last Updated : 23 Jul, 2025

C++ supports function overloading, i.e., there can be more than one function with the same name but, different parameters. **How does the C++ compiler distinguish between different functions when it generates object code** - it changes names by adding information about arguments. This technique of adding additional information to function names is called **Name Mangling**. C++ standard doesn't specify any particular technique for name mangling, so different compilers may append different information to function names.

Дальше больше: Статические библиотеки

- Проблема: Передавать сотни .o файлов компоновщику неудобно. Хранить копии стандартных функций (например, libc) в каждом исполняемом файле расточительно.

- Решение: Статическая библиотека (архив, .a в Linux).

Дальше больше: Статические библиотеки

- Проблема: Передавать сотни .o файлов компоновщику неудобно. Хранить копии стандартных функций (например, libc) в каждом исполняемом файле расточительно.

- Решение: Статическая библиотека (архив, .a в Linux).

- **Однако:** статическая компоновка включает код используемой библиотеки в исполняемый файл. Это увеличивает его размер и означает, что обновление библиотечного кода обычно требует пересборки программы.

Алгоритм разрешения ссылок

- Ключевой принцип: Обработка входных файлов строго слева направо.

- Состояние компоновщика:
  - E: множество .o файлов, которые войдут в итоговый файл.
  - U: множество неразрешенных ссылок (UND).
  - D: множество уже определенных символов.

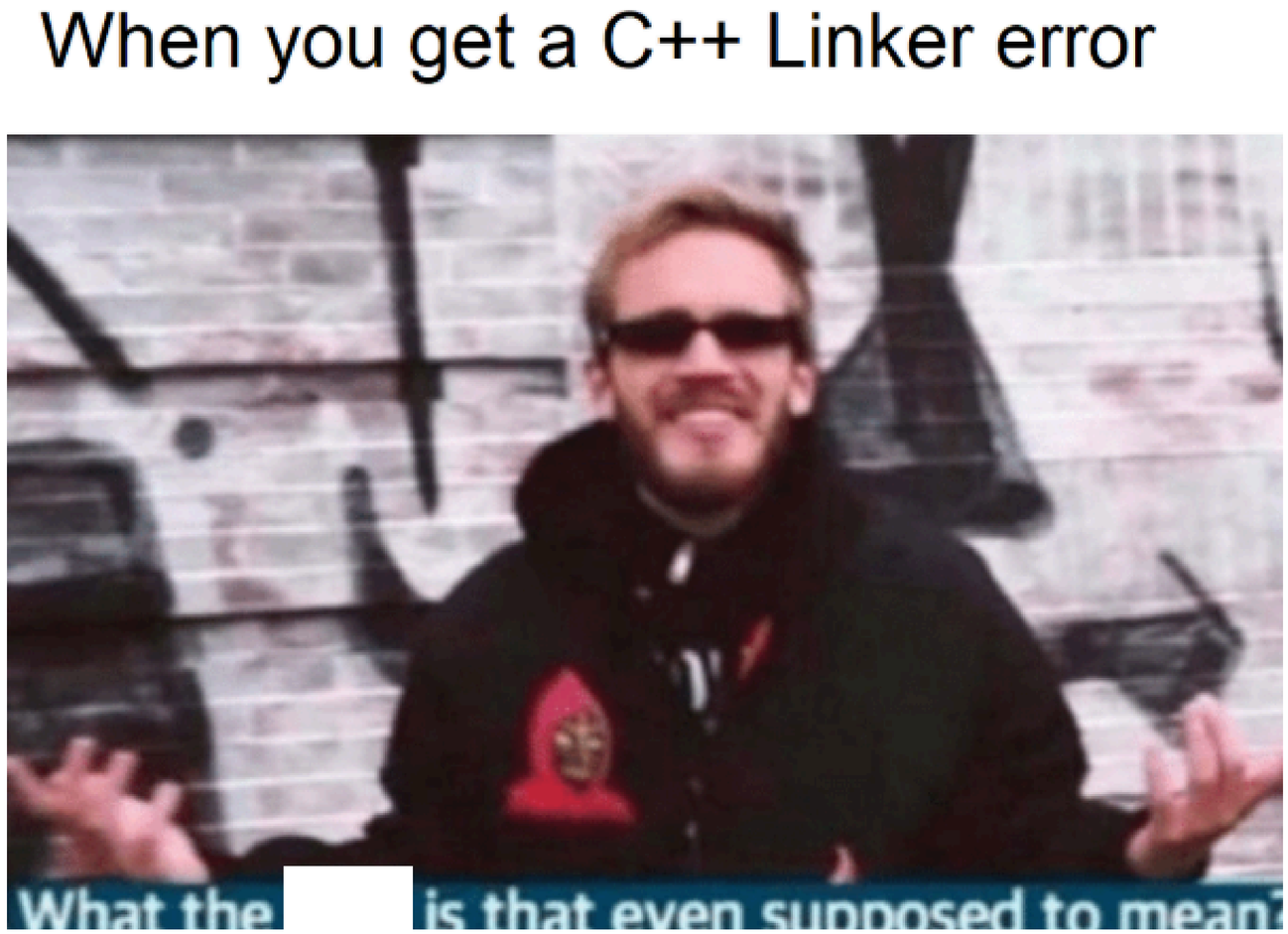
- При встрече архива: компоновщик извлекает из него только те .o файлы, которые содержат определения для символов, текущих находящихся в U.

Алгоритм разрешения ссылок

- Сценарий А (Правильно): gcc main.o -L. -lmylib
  a. Видит main.o, добавляет в E, видит UND на my_func, добавляет в U.
  b. Видит -lmylib, находит my_func в архиве, извлекает mylib.o, добавляет в E, U очищается.

- Сценарий Б (Ошибка): gcc -L. -lmylib main.o
  a. Видит -lmylib. U пусто. Ничего не извлекает из архива.
  b. Видит main.o. Добавляет в E, видит UND на my_func.
  c. Конец ввода. U не пусто → Ошибка: undefined reference to 'my_func'.

Современный контекст



Современный контекст

- Эволюция компиляторов: Начиная с GCC 10 (2020 г.), флаг -fno-common включен по умолчанию.
- Что это дает: Все неинициализированные глобальные переменные теперь считаются сильными.
- Связь с C++: One Definition Rule (ODR). Использование inline переменных или анонимных namespace для решения тех же проблем на уровне языка.